towards ECMAScript Standards (the scripting language specification standardised by ECMA International in ECMA-262 and ISO/IEC 16262). The detailed documentation on Haxe programming is available for download on *https://github.com/HaxeFoundation/HaxeManual/raw/master/HaxeManual/HaxeManual.pdf*. The Haxe source code can be entered using any text editor. Once the program is entered, it can be compiled into the available target programming languages/bytecode using the Haxe cross-compiler. A sample Hello World program in Haxe is as shown below:

```
class HelloWorld {
    static public function main() {
        trace("Welcome to Haxe Programming");
    }
}
```

This code should be saved as *HelloWorld.hx*. Then, from the command prompt it will be cross-compiled into JavaScript using the following command:

```
haxe -main HelloWorld -js HelloWorld.js
```

The compilation will result in the creation of the JavaScript source file *HelloWorld.js*, the contents of which are shown below:

```
(function (console) { "use strict";
var HelloWorld = function() { };
HelloWorld.main = function() {
    console.log("Welcome to Haxe Programming");
};
HelloWorld.main();
})(typeof console != "undefined" ? console : {log:function()
{}});
```

The same Haxe code will be compiled into Python using the corresponding option in the Haxe compiler, as follows:

```
    :
haxe -main HelloWorld -python HelloWorld.py
```

This would generate the Python source code as output in the file *HelloWorld.py*, the contents of which are shown below:

```
class HelloWorld:
    @staticmethod
    def main():
      print("Welcome to Haxe Programming")
HelloWorld.main()
```

With the default installation of Haxe in the Windows environment, when you try to target certain programming languages like C#, there will be an error message like:

```
"Error:Library hxcs is not installed".
```

This can be rectified by using the following command:

```
haxelib install hxcs
```

After this installation, the source program will be compiled into the C# target with the following command:

```
haxe -main HelloWorld -cs HelloWorldCS
```

The point to be noted here is that 'HelloWorldCS' is a folder name, in contrast to the file names supplied for Python and JavaScript compilation. The above command creates a folder and places all the necessary files and sub-directories for the C# project. Another interesting feature is that the generated C# code is also compiled and an 'exe' in the name 'HelloWorld.exe' is built in the 'bin' folder, which will be readily executed.

The Haxe standard library

The standard library provided with Haxe has support for various commonly used tasks. It has three major categories as shown in Figure 3.

- The general-purpose component of the standard library covers features like arrays, strings, regular expressions and XML. It also covers advanced features like various encryption algorithms (*haxe.crypto*, *haxe.JSON*, etc).
- The system component has features like DB manipulation, file handling, process handling, etc.
- The target-specific library component covers features specific to the corresponding target languages.

Haxe based frameworks and tools

Haxe has good support for various domains as shown in Figure 4. There are various Haxe based tools which assist in the above-specified application domains:

- OpenFL
- Flambe
- Nape Physics Engine

The OpenFL framework is built using Haxe and

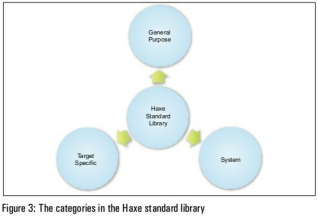

Figure 3: The categories in the Haxe standard library